

# MIXSV: Mixup with Shapley Values

Jinfei Liu\*, Xiaokai Mao\*, Hanxi Liu\*, Junyuan Pang\*, Qiheng Sun\*<sup>†</sup>, Kui Ren\*

\*Zhejiang University, Hangzhou, China

<sup>†</sup>The Hong Kong Polytechnic University, Hong Kong, China

{jinfeiliu, xiaokaimao, liuhx01, junyuanpang, kuiren}@zju.edu.cn

{qiheng.sun}@polyu.edu.hk

**Abstract**—Improving machine learning model performance often relies on effective data augmentation. Among them, Mixup, a strategy that generates new data points by interpolating between pairs of existing ones, has garnered significant attention for its simplicity and effectiveness. However, traditional Mixup treats all data points equally, ignoring their varying importance, which may result in generated data points that are less relevant to tasks and of low quality. To address this, we propose MIXSV, a Shapley-guided Mixup method that assigns interpolation weights according to the contribution of each data point. For each sampled pair, MIXSV shifts and normalizes their Shapley values and uses them as the shape parameters of a Beta distribution, so that higher-value samples receive larger expected weights. To make the method practical, we use kNN Shapley as an efficient proxy for data valuation. We further provide a theoretical analysis showing that Shapley-guided interpolation can reduce validation-distribution discrepancy and attenuate the effect of corrupted labels under natural alignment conditions. Experiments on synthetic and real-world datasets show that MIXSV improves classification accuracy, distribution alignment, and robustness against adversarial perturbations and label poisoning compared with standard Mixup and its variants.

**Index Terms**—data Shapley, data management

## I. INTRODUCTION

The quality of the training data plays a pivotal role in determining the performance of a machine learning model. Data augmentation techniques, including Cutout [1], Mixup [2], and CutMix [3], enhance generalization and robustness by introducing diversity into the training data, leading to improved model performance. Mixup, introduced by Zhang et al. [2], has attracted significant attention as a simple yet effective data augmentation strategy. The method involves generating new training data points by convexly combining pairs of features with their corresponding labels, effectively improving generalization and making the model robust against adversarial techniques [4].

Mixup relies on random interpolation weights sampled from a Beta distribution ( $\lambda \sim \text{Beta}(\alpha, \alpha)$ ), treating all data points as equally important. However, the assumption may not hold in real-world datasets especially when noisy or mislabeled data points are present. For instance, mixing a high quality data point and a mislabeled or noisy data point can produce a misleading mixed data point, confusing the model and degrading its performance. Even for clean and correctly labeled data points, their importance can vary based on factors such as their representativeness of the underlying data distribution. Figure 1a shows a case of distribution shift where Mixup does

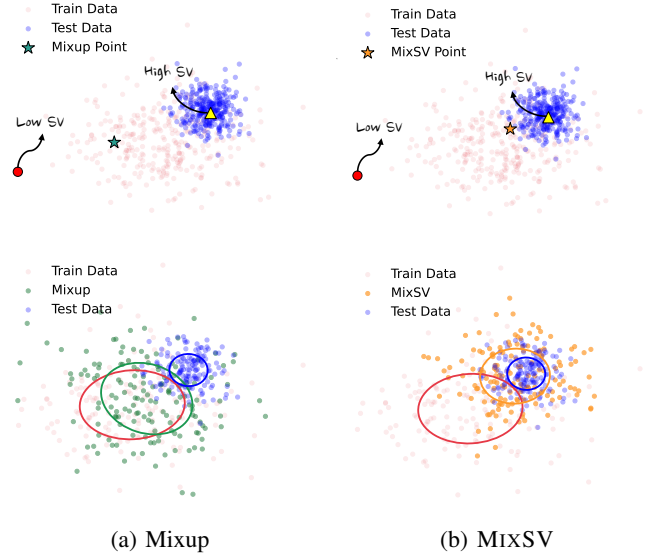


Fig. 1: Comparison of Mixup and MIXSV in aligning training, mixed, and test distributions. MIXSV reduces the distributional shift between mixed and test data.

not consider the importance of the involved data points. The interpolation weights have an expected value of 0.5 and the generated Mixup point (green star) lies in the middle between the two original data points (red circle and yellow triangle). When Mixup is applied across the entire dataset, the resulting mixed dataset (green points) exhibits a significant shift from the distribution of the test set (blue points). Such challenges call for a more principled approach to Mixup that considers the inherent quality and importance of individual data points. A key question arises: *Can we design a novel mixup method that takes into account the importance of individual data points?*

The Shapley value [5], named in honor of Lloyd Shapley, is a cornerstone method in cooperative game theory for fair contribution allocation that satisfies four desired properties: *Balance*, *Symmetry*, *Zero element*, and *Additivity*. These properties make the Shapley value a theoretically grounded approach for evaluating contributions in complex systems. In the context of data valuation, the Shapley value has been widely adopted to quantify the importance of each data point by measuring its contribution to model performance [6], [7]. Data valuation scores derived from the Shapley value allow us to rank and

prioritize data points based on their impact on the model.

However, while Shapley value-based data valuation offers significant insights into the importance of individual data points, their potential in guiding Mixup remains underexplored. By integrating Shapley value-based data valuation into Mixup, we can dynamically adjust interpolation weights, enabling a more principled approach that prioritizes high-quality data points and mitigates the influence of noisy or less important data.

**Contribution.** In this paper, we propose MIXSV (Mixup with Shapley Values), a novel augmentation strategy that leverages Shapley values to replace the random weighting mechanism in Mixup. Unlike the traditional Mixup, which uses random coefficients for data interpolation, MIXSV dynamically assigns weights to data points based on their Shapley values, reflecting their contributions to the prediction task. This approach mitigates the noise introduced by random mixing weights in traditional Mixup, ensuring that high-quality data points play a more significant role in the training process while reducing the influence of noisy data. As illustrated in Figure 1b, MIXSV assigns greater weight to the more important data point (yellow triangle), producing a mixed point (orange star) closer to it. Compared to Mixup, MIXSV reduces the distributional shift between the mixed dataset (orange points) and the test dataset (blue points). However, calculating Shapley values is known as #P-hard for general tasks. To address this challenge, we leverage the lazy learning nature of kNN, enabling Shapley values to be computed in polynomial time for kNN-based tasks. By employing kNN Shapley as a proxy for neural network tasks, we reduce computational complexity significantly while only losing small performance. We briefly summarize our contributions as follows:

- We propose MIXSV, a novel augmentation method that assigns interpolation weights based on the contribution of each data point to model performance. Our theoretical analysis shows that MIXSV generates augmented data distributions closer to the test distribution, ensuring more effective training.
- We propose to use kNN Shapley instead of using sampling methods to calculate Shapley values to reduce the computational complexity, so that MIXSV can be used for large datasets and neural networks
- Extensive experiments on both synthetic and real-world datasets (e.g., Adult, Arrhythmia, and CIFAR-100) show that MIXSV consistently outperforms existing Mixup strategies, achieving up to 3.73% gains in generalization accuracy and up to 2.62% higher than other baselines in data poisoning.

## II. RELATED WORK

Data augmentation has become a cornerstone technique in machine learning, enabling models to generalize better by artificially expanding training datasets [8]. Among various augmentation strategies, Mixup and its numerous variants have gained significant attention for their ability to improve

model robustness by interpolating data points and their corresponding labels. Importantly, Mixup is orthogonal to other data augmentation methods, meaning it can be effectively combined with techniques such as random cropping, flipping, or CutMix [9], further enhancing model performance [10]. For instance, integrating Mixup with spatial transformations has been shown to yield complementary benefits in vision tasks. Motivated by the unique potential of Mixup, we focus on advancing its framework to unlock further improvements in generalization and robustness.

Methods such as Mixup [2], CutMix [9], and Manifold Mixup [11] generate mixed data by mixing existing data points. These techniques generally assume that all training data contribute equally to the learning process, overlooking the varying importance of individual data points. Some recent advancements have sought to address these limitations by incorporating adaptive or saliency-based strategies. For instance, AdaMixup [12] introduces an adaptive approach to adjust the mixing coefficient, but it is just designed to avoid manifold intrusion. On the other hand, saliency-based Mixup methods [13]–[15] utilize saliency maps to guide the mixing process by emphasizing pixel-level importance. While effective for image data, these approaches are inherently limited by their reliance on spatial saliency, making them less applicable to non-image domains.

Shapley value, derived from cooperative game theory, is a principled and fair method for quantifying the contribution of individual elements in a system. It ensures fairness by attributing contributions based on marginal gains, making it a powerful tool for various applications. In the context of data valuation, methods like Data Shapley [6] leverage the Shapley value to assess the importance of individual data points to model performance. Beyond data valuation, the Shapley value has also found extensive applications in domains such as data explanations [16], data pricing [17], and feature selection [18], demonstrating its versatility and effectiveness.

The most related work to ours is the integration of Shapley values into data augmentation strategies. For instance, Shapley-Based Feature Augmentation (SFA) [19] improves predictive performance on tabular data by leveraging feature importance values derived from Shapley explanations. Similarly, a Shapley value-based Mixup approach for text data [20] utilizes word-level importance scores to refine label assignments, enhancing semantic consistency. Shap-Mix [21], on the other hand, incorporates Shapley values for skeleton-based action recognition, particularly improving performance on tail classes. However, these methods primarily focus on feature-level interpolation within single instances rather than exploring cross-data point mixing, as our method does.

## III. PRELIMINARIES

In this section, we briefly review the basic concepts used in MIXSV. For reference, Table I summarizes the frequently used notations. Section III-A introduces standard Mixup and its interpolation rule for generating mixed samples. Section III-B

TABLE I: Summary of frequently used notations.

| Notation               | Definition                                                      |
|------------------------|-----------------------------------------------------------------|
| $n$                    | the size of the training dataset                                |
| $z_i$                  | the $i^{th}$ data point in the training set                     |
| $\tilde{x}$            | the mixed input generated by Mixup or MIXSV                     |
| $\tilde{y}$            | the mixed label generated by Mixup or MIXSV                     |
| $\lambda$              | the interpolation coefficient for mixing two data points        |
| $\alpha$               | the Mixup hyperparameter controlling the interpolation strength |
| $\mathcal{U}(\cdot)$   | the utility function used to evaluate a coalition               |
| $\mathcal{SV}_i$       | the Shapley value of data point $z_i$                           |
| $\hat{\mathcal{SV}}_i$ | the shifted and normalized Shapley value of $z_i$               |
| $B$                    | the mini-batch size                                             |
| $K$                    | the number of neighbors used in kNN Shapley                     |
| $p_{ij}$               | the expected interpolation weight assigned to $x_i$ in MIXSV    |
| $c_y(x)$               | the validation discrepancy of point $x$ for class $y$           |

reviews Shapley values and explains how they measure the contribution of each training point to model utility.

#### A. Mixup

Suppose there exists a joint probability distribution  $P(x, y)$  across sets  $\mathcal{X}$  and  $\mathcal{Y}$ , along with a training set  $\mathcal{S}_{trn} = \{(x_1, y_1), \dots, (x_n, y_n)\}$ , where  $x_i \in \mathcal{X} \subseteq \mathbb{R}^p$  and  $y_i \in \mathcal{Y} \subseteq \mathbb{R}^q$  are independently and identically distributed (i.i.d.) samples from this joint distribution. The fundamental concept behind Mixup involves the generation of novel training data points through the mixing of two randomly chosen data points from training set. Given two data points  $(x_i, y_i)$  and  $(x_j, y_j)$ , where  $x_i$  and  $x_j$  represent input features and  $y_i$  and  $y_j$  denoting their respective labels, a new generated training data point  $(\tilde{x}, \tilde{y})$  is formed in the following manner:

$$\begin{aligned}\tilde{x} &= \lambda x_i + (1 - \lambda)x_j, \\ \tilde{y} &= \lambda y_i + (1 - \lambda)y_j,\end{aligned}$$

where  $\lambda \in [0, 1]$  is a mixing coefficient drawn from a Beta distribution  $\lambda \sim \text{Beta}(\alpha, \alpha)$  and  $\alpha$  is a hyperparameter that controls the strength of the interpolation.

#### B. Shapley values

Consider a set of data points  $\mathcal{D} = \{z_1, \dots, z_n\}$ . A coalition  $\mathcal{S}$  is a subset of  $\mathcal{D}$  that cooperates to complete an ML task, for instance, training an ML model. A utility function  $\mathcal{U}(\mathcal{S})$  ( $\mathcal{S} \subseteq \mathcal{D}$ ) is the utility of a coalition  $\mathcal{S}$  for an ML task, which is typically the classification accuracy of the model trained on  $\mathcal{S}$ . The marginal contribution of data point  $z_i$  with respect to a coalition  $\mathcal{S}$  ( $z_i \notin \mathcal{S}$ ) is  $\mathcal{U}(\mathcal{S} \cup \{z_i\}) - \mathcal{U}(\mathcal{S})$ . The Shapley value  $\mathcal{SV}_i$  measures the expectation of marginal contribution by data point  $z_i$  in all possible coalitions over  $\mathcal{D}$ . That is,

$$\mathcal{SV}_i(\mathcal{U}) = \frac{1}{n} \sum_{\mathcal{S} \subseteq \mathcal{N} \setminus \{z_i\}} \frac{\mathcal{U}(\mathcal{S} \cup \{z_i\}) - \mathcal{U}(\mathcal{S})}{\binom{n-1}{|\mathcal{S}|}} \quad (1)$$

Computing the exact Shapley value has to enumerate the utilities of all coalitions which is proved to be #P-hard [22]. For convenience, we will rewrite  $\mathcal{SV}_i(\mathcal{U})$  as  $\mathcal{SV}_i$  in the following sections.

## IV. THE ALGORITHM OF MIXSV

In this section, we introduce the proposed MIXSV strategy. Section IV-A provides an overview of the core motivation and methodology behind MIXSV. In Section IV-B, we formally present the mathematical formulation of MIXSV, illustrating how Shapley values parameterize Beta distributions to control interpolation weights dynamically. Section IV-C addresses the computational complexity by leveraging kNN Shapley.

#### A. Overview

Traditional Mixup randomly samples the interpolation coefficient  $\lambda \sim \text{Beta}(\alpha, \alpha)$ , treating all data points equally. This ignores the varying importance of data points, leading to potential issues when noisy or mislabeled data are present, which can confuse the model.

MIXSV addresses this limitation by leveraging Shapley values to dynamically adjust  $\lambda$  based on each data point's contribution to model performance. Instead of fixed randomness, MIXSV parameterizes the Beta distribution using Shapley values, giving greater weight to more important data points. This targeted weighting reduces the influence of noisy data while enhancing the overall effectiveness of Mixup.

#### B. Mixup with Shapley Values

Instead of sampling  $\lambda$  from a fixed Beta distribution, MIXSV draws  $\lambda$  from a Beta distribution whose shape parameters are determined by the Shapley values of the two data points:

$$\lambda \sim \text{Beta}(\hat{\mathcal{SV}}_i, \hat{\mathcal{SV}}_j), \quad \text{where } \hat{\mathcal{SV}}_i + \hat{\mathcal{SV}}_j = 2\alpha. \quad (2)$$

The specific choice of maintaining parameter summation at  $2\alpha$  achieves two essential objectives: 1) It naturally recovers the original  $\text{Beta}(\alpha, \alpha)$  distribution when Shapley values are symmetric, ensuring backward compatibility; 2) The overall “scale” of the Beta distribution to be consistent with parameterization of standard Mixup. This lets our method be compared with standard Mixup more directly, since both share the same total shape parameter;

It is worth noting that Shapley values may be negative, reflecting the fact that a data point's marginal contribution can be detrimental in certain coalitions. However, a negative Shapley value does not necessarily imply that the data point is globally harmful when included in the full training set. For instance, in cooperative game theory, it is possible for a player to have a negative Shapley value even though their presence improves the overall utility in some contexts. Thus, in MIXSV, rather than discarding such points, we shift and normalize all Shapley values to ensure valid Beta distribution parameters and retain their nuanced contributions in a probabilistically principled way.

In practice, raw Shapley values  $\mathcal{SV}_i$  can be negative, making them unsuitable as Beta distribution parameters. To resolve this, we apply a pairwise shift approach: For each pair of data points (i,j) selected for MIXSV, we first extract their original Shapley values  $\mathcal{SV}_i, \mathcal{SV}_j$ . We then shift both values upward if the minimum of these two values is negative, ensuring both

become strictly positive:  $\mathcal{SV}'_i = \mathcal{SV}_i - \min(\mathcal{SV}_i, \mathcal{SV}_j) + \epsilon$ ,  $\mathcal{SV}'_j = \mathcal{SV}_j - \min(\mathcal{SV}_i, \mathcal{SV}_j) + \epsilon$  where  $\epsilon$  is a small constant (e.g.,  $10^{-6}$ ) added to avoid zeros. Finally, we normalize the shifted pairwise values so that their sum equals  $2\alpha$ :  $\hat{\mathcal{SV}}_i = \frac{\mathcal{SV}'_i}{\mathcal{SV}'_i + \mathcal{SV}'_j} \times 2\alpha$ ,  $\hat{\mathcal{SV}}_j = \frac{\mathcal{SV}'_j}{\mathcal{SV}'_i + \mathcal{SV}'_j} \times 2\alpha$ . The pairwise shift is used because Mixup is performed on pairs. It preserves the relative ordering between the two selected points while avoiding invalid non-positive Beta parameters. The following normalization keeps the total concentration fixed at  $2\alpha$ , so  $\alpha$  still controls the interpolation strength as in standard Mixup. These normalized and strictly positive Shapley values are then used as the Beta distribution shape parameters in MIXSV, ensuring valid parameterizations and stable sampling. We also explored an alternative approach that applies different transformations to negative Shapley values before normalization. Specifically, we evaluated three transformations: linear shifting (current method), softmax normalization, and exponential shifting, as shown in Table II. Our experiments indicate that linear shifting consistently achieves the best performance across datasets, justifying its selection as the primary method in MIXSV.

TABLE II: Comparison of methods for handling negative Shapley values (classification accuracy in %).

| Dataset    | Linear       | Softmax      | Exponential |
|------------|--------------|--------------|-------------|
| synthetic  | 71.00        | <b>73.00</b> | 69.33       |
| adult      | <b>86.61</b> | 84.69        | 84.42       |
| cifar100   | <b>71.30</b> | 69.08        | 69.30       |
| arrhythmia | <b>67.76</b> | 65.93        | 67.03       |

When the normalized Shapley values  $\mathcal{SV}_i$  and  $\mathcal{SV}_j$  are used as the shape parameters for the Beta distribution, they not only determine the interpolation weighting but also directly influence the expected value of  $\lambda$ . With the constraint  $\mathcal{SV}_i + \mathcal{SV}_j = 2\alpha$ , the expected value of  $\lambda$  is given by:

$$\mathbb{E}[\lambda] = \frac{\mathcal{SV}_i}{\mathcal{SV}_i + \mathcal{SV}_j} = \frac{\mathcal{SV}_i}{2\alpha}. \quad (3)$$

For instance, consider  $\alpha = 1$ , with original Shapley values  $\mathcal{SV}_i = 4$  and  $\mathcal{SV}_j = 1$ . After normalization,  $\mathcal{SV}_i = 1.6$  and  $\mathcal{SV}_j = 0.4$ . Consequently,  $\lambda$  will be sampled from  $Beta(1.6, 0.4)$ .

As shown in Figure 2, when  $\mathcal{SV}_i > \mathcal{SV}_j$ , the Beta distribution becomes asymmetric, resulting in the expected value of  $\lambda$  to be more biased towards  $\mathcal{SV}_i$ . This implies that the interpolation favors the data point associated with  $\mathcal{SV}_i$ , giving it greater influence in the augmented data point.

This adaptive mechanism which is reflected by Shapley values of data points leverages the informativeness carried by value to adjust the interpolation process. Data points with higher  $\mathcal{SV}_i$  not only dominate the interpolation but also guide the augmented data points toward regions that better represent the underlying data distribution. Simultaneously, the reduced weight on less informative data points mitigates the impact of noise.

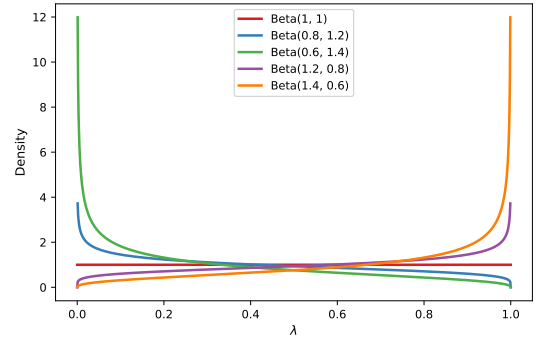


Fig. 2: Illustration of Beta distributions with varying shape parameters

---

#### Algorithm 1: MixSV

---

**Input:** Mini-batch inputs  $\{z_i = (x_i, y_i)\}_{i=1}^B$ , and their Shapley values  $\mathcal{SV}_i$  for  $1 \leq i \leq B$ .  
**Output:** Mixed inputs  $\tilde{X}$ , mixed targets  $y_a, y_b$ , and mixing coefficients  $\lambda$ .

- 1  $B \leftarrow$  batch size;
- 2 Generate permutation  $\pi \sim \text{Shuffle}([1, B])$ ;  
// Random pairing
- 3  $\epsilon \leftarrow 10^{-6}$ ;
- 4 **for**  $i \leftarrow 1$  **to**  $B$  **do**
- 5    $j \leftarrow \pi[i]$ ;
- 6    $\text{min\_pair} \leftarrow \min(\mathcal{SV}_i, \mathcal{SV}_j)$ ;
- 7    $\text{offset}_i \leftarrow \max(-\text{min\_pair} + \epsilon, 0)$ ;  
// Negativity correction
- 8    $\hat{\mathcal{SV}}_i \leftarrow \mathcal{SV}_i + \text{offset}_i$ ;                               // Positive shifting
- 9    $\hat{\mathcal{SV}}_j \leftarrow \mathcal{SV}_j + \text{offset}_i$ ;
- 10    $\alpha_i \leftarrow \frac{\hat{\mathcal{SV}}_i}{\hat{\mathcal{SV}}_i + \hat{\mathcal{SV}}_j} \times 2\alpha$ ;                               // Normalized parameters
- 11    $\beta_i \leftarrow \frac{\hat{\mathcal{SV}}_j}{\hat{\mathcal{SV}}_i + \hat{\mathcal{SV}}_j} \times 2\alpha$ ;
- 12    $\lambda_i \sim \text{Beta}(\alpha_i, \beta_i)$ ;
- 13    $\tilde{x}_i \leftarrow \lambda_i x_i + (1 - \lambda_i) x_{\pi[i]}$ ;
- 14 **return**  $\tilde{X}, y, y[\pi], \lambda$ ;

---

The detailed algorithm is shown in Algorithm 1. MIXSV takes a mini-batch of data points  $\{z_i = (x_i, y_i)\}_{i=1}^B$  and their corresponding Shapley values  $\mathcal{SV}_i$  as input. The algorithm begins by generating a random permutation of the batch indices to pair data points. For each pair, we inspect  $\mathcal{SV}_i$  and  $\mathcal{SV}_{\pi(i)}$ . If any value is negative, a small positive shift is applied to both so that they all become nonnegative. We then normalize the shifted Shapley values so that their sum for each pair equals  $2\alpha$ . These adjusted Shapley values are then used to parameterize a Beta distribution, from which the mixing coefficient  $\lambda$  is sampled. Finally, mixed inputs  $\tilde{x}$  are created

<sup>1</sup>Similar to the settings in mixup, the mix of labels is reflected in the loss function.

by linearly interpolating between the paired data points, and their labels  $y_a$  and  $y_b$  are stored for supervised training.

### C. Mixup with kNN Shapley Values

We precompute one Shapley value for each training point before Mixup training. The Shapley values are computed using the training set and a clean validation set, and then reused during all training epochs. During each mini-batch, MIXSV retrieves the precomputed values of the sampled examples and uses them to construct pair-specific Beta parameters. Intuitively, when calculating the Shapley value of a data point, we use the same machine learning model and dataset as the target task and perform the same training process. However, calculating the exact Shapley value for each data point is computationally prohibitive, as it requires evaluating the contribution of the point across all possible subsets of the training data, with a time complexity of  $O(2^N)$ . The Monte Carlo sampling method provides an unbiased approximate estimate of the Shapley value. However, due to its inherent complexity, it can only be used for traditional machine learning models, and is still impossible to use when deep neural networks are involved.

To overcome this challenge, we utilize kNN Shapley [23], an exact method specifically designed for unweighted k-nearest neighbor (kNN) classifiers and regressors. This approach leverages the structure of kNN models to reduce computational complexity to  $O(N \log N)$ , representing an exponential improvement over traditional methods while maintaining accuracy.

Currently, kNN Shapley can only compute Shapley values for tabular datasets, our proposed MIXSV is likewise limited to tabular data. (Of course, one can adopt the Shapley Value Algorithm Based on Monte Carlo to extend MIXSV to image datasets, but these approaches typically require significantly longer running times; we report some of our findings from these experiments in the Appendix.). Nonetheless, as more general and efficient Shapley value computation methods for deep learning models emerge, MIXSV could be extended to a broader range of data modalities.

To empirically validate the use of kNN Shapley as a surrogate for Monte Carlo Shapley estimates, we conduct a correlation analysis using an SVM as the utility model. Specifically, we compute Shapley values with both Monte Carlo sampling using 10,000 iterations and kNN Shapley on the same subsets of training points. Figure 3 shows scatter plots on two representative tabular datasets, SYNTHETIC and ADULT. Each point represents one training sample, with its x-axis value given by kNN Shapley and its y-axis value given by Monte Carlo Shapley. The two methods show strong agreement, with Pearson correlations of  $r = 0.764$  on SYNTHETIC and  $r = 0.899$  on ADULT. These results provide empirical support for using kNN Shapley as an efficient proxy for Monte Carlo-based Shapley values in our experiments.

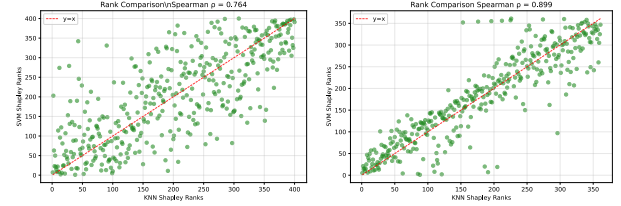


Fig. 3: Scatter plots of kNN Shapley versus Monte Carlo Shapley with 10,000 samples for 2,000 points each from the SYNTHETIC and ADULT datasets. The dashed lines denote equality. Pearson correlations are  $r = 0.764$  and  $r = 0.899$ , respectively.

## V. THEORETICAL ANALYSIS

This section provides an upper-bound explanation for why MIXSV can generate more useful mixed samples than standard Mixup. First, Section V-A shows that, when Shapley-guided weights are aligned with validation closeness, MIXSV reduces an upper bound on the discrepancy between mixed samples and the clean validation distribution. Second, Section V-B analyzes label-noise attenuation and proves that MIXSV assigns smaller expected weight to corrupted labels when clean samples receive larger Shapley values than mislabeled samples. Finally, Section V-C studies the local ordering property of kNN-Shapley and explains why kNN-Shapley can give larger values to clean and label-matching samples near the validation distribution.

### A. Validation-discrepancy Alignment

We first study how MIXSV changes the location of mixed samples relative to the validation distribution. Let  $\mathcal{X} \subseteq \mathbb{R}^p$  be a convex feature space. For each class  $y$ , define the validation discrepancy of a point  $x$  as

$$c_y(x) = \mathbb{E}_{V \sim \mu_v^y} [\|x - V\|].$$

The empirical version of  $c_y(x)$  is

$$\hat{c}_y(x) = \frac{1}{|D_v^y|} \sum_{v \in D_v^y} \|x - v\|.$$

This quantity measures how far  $x$  is from the validation distribution of the same class. It also gives an upper bound on the conditional Wasserstein distance. For any distribution  $\nu^y$  over class- $y$  samples,

$$W_1(\nu^y, \mu_v^y) \leq \mathbb{E}_{X \sim \nu^y} [c_y(X)]. \quad (4)$$

This follows by using the independent coupling between  $X \sim \nu^y$  and  $V \sim \mu_v^y$ .

Now consider a same-label pair  $(x_i, y)$  and  $(x_j, y)$ . MIXSV generates

$$\tilde{x}_{ij} = \lambda x_i + (1 - \lambda) x_j, \quad \lambda \sim \text{Beta}(\alpha_{ij}, \beta_{ij}),$$

where the two Beta parameters are obtained from the pairwise shifted and normalized Shapley values. Let

$$p_{ij} = \mathbb{E}[\lambda] = \frac{\alpha_{ij}}{\alpha_{ij} + \beta_{ij}}. \quad (5)$$

Since MIXSV normalizes the two Beta parameters so that  $\alpha_{ij} + \beta_{ij} = 2\alpha$ , the value of  $p_{ij}$  is exactly the expected interpolation weight assigned to  $x_i$ .

We use the following alignment condition.

**Assumption V.1.** For a same-label pair  $(x_i, y)$  and  $(x_j, y)$ , the expected MIXSV weight is aligned with validation discrepancy:

$$\left(p_{ij} - \frac{1}{2}\right) (c_y(x_i) - c_y(x_j)) \leq 0. \quad (6)$$

This assumption means that MIXSV gives a larger expected weight to the endpoint that is closer to the validation distribution. For example, if  $c_y(x_i) < c_y(x_j)$ , then  $x_i$  is more aligned with the validation distribution, and the condition requires  $p_{ij} \geq 1/2$ . This is consistent with the goal of using Shapley values as data-importance scores.

**Theorem V.1.** Consider a same-label pair  $(x_i, y)$  and  $(x_j, y)$ . Suppose Assumption V.1 holds. Then the mixed sample generated by MIXSV satisfies

$$\mathbb{E}_\lambda [c_y(\tilde{x}_{ij})] \leq \frac{c_y(x_i) + c_y(x_j)}{2} - \left|p_{ij} - \frac{1}{2}\right| |c_y(x_i) - c_y(x_j)|. \quad (7)$$

In particular,  $\mathbb{E}_\lambda [c_y(\tilde{x}_{ij})] \leq \frac{c_y(x_i) + c_y(x_j)}{2}$ .

*Proof.* Since  $\mathcal{X}$  is convex and  $c_y(x)$  is an expectation of convex functions  $\|x - V\|$ , the function  $c_y$  is convex. Thus,

$$c_y(\lambda x_i + (1 - \lambda)x_j) \leq \lambda c_y(x_i) + (1 - \lambda)c_y(x_j).$$

Taking expectation over  $\lambda$  gives

$$\mathbb{E}_\lambda [c_y(\tilde{x}_{ij})] \leq p_{ij}c_y(x_i) + (1 - p_{ij})c_y(x_j).$$

The right-hand side can be rewritten as

$$\frac{c_y(x_i) + c_y(x_j)}{2} + \left(p_{ij} - \frac{1}{2}\right) (c_y(x_i) - c_y(x_j)).$$

By Assumption V.1,

$$\left(p_{ij} - \frac{1}{2}\right) (c_y(x_i) - c_y(x_j)) = -\left|p_{ij} - \frac{1}{2}\right| |c_y(x_i) - c_y(x_j)|.$$

Substituting this equality into the previous bound proves the result.  $\square$

Theorem V.1 shows that MIXSV reduces a pairwise upper bound on the validation discrepancy whenever Shapley-based weights are aligned with validation closeness. The improvement term  $|p_{ij} - \frac{1}{2}| |c_y(x_i) - c_y(x_j)|$  has a clear meaning. MIXSV gives a larger gain when the two endpoints have very different validation discrepancies and the Shapley-guided weight is far from the uniform value  $1/2$ . When the two endpoints are equally useful, or when their Shapley values are nearly equal, MIXSV reduces to a behavior close to standard Mixup.

We can also state the result at the distribution level. Let  $\tilde{\mu}_t^y$  be the distribution of same-label mixed samples generated

by MIXSV for class  $y$ . Let  $\bar{\mu}_t^y$  be the distribution obtained by drawing one endpoint uniformly from the same sampled pairs. If Assumption V.1 holds for all sampled same-label pairs, then

$$\mathbb{E}_{\tilde{X} \sim \tilde{\mu}_t^y} [c_y(\tilde{X})] \leq \mathbb{E}_{X \sim \bar{\mu}_t^y} [c_y(X)]. \quad (8)$$

Therefore,

$$W_1(\tilde{\mu}_t^y, \bar{\mu}_t^y) \leq \mathbb{E}_{\tilde{X} \sim \tilde{\mu}_t^y} [c_y(\tilde{X})] \leq \mathbb{E}_{X \sim \bar{\mu}_t^y} [c_y(X)]. \quad (9)$$

This result makes explicit why Shapley-guided interpolation can improve distribution alignment: high-value samples pull the generated samples toward regions that are closer to the clean validation distribution.

### B. Label-noise Attenuation

We next analyze how MIXSV reduces the influence of mislabeled training points. This analysis applies to arbitrary pairs, including pairs from different classes. Let  $y_i^*$  be the clean one-hot label of  $x_i$ , and let  $\tilde{y}_i$  be its observed label used for training. Define the corruption indicator  $r_i = \mathbb{1}[\tilde{y}_i \neq y_i^*]$ . Thus,  $r_i = 0$  means that the observed label is clean, and  $r_i = 1$  means that the observed label is corrupted. For a pair  $(i, j)$ , let

$$a_i^{(ij)} = \mathcal{SV}_i + \delta_{ij}, \quad a_j^{(ij)} = \mathcal{SV}_j + \delta_{ij},$$

where  $\delta_{ij} = \max\{-\min(\mathcal{SV}_i, \mathcal{SV}_j) + \epsilon, 0\}$ . Then the expected MIXSV interpolation weight assigned to  $x_i$  is  $p_{ij} = \frac{a_i^{(ij)}}{a_i^{(ij)} + a_j^{(ij)}}$ . For the same pair  $(i, j)$ , MIXSV constructs the mixed input and observed mixed label as

$$\tilde{x}_{ij} = \lambda x_i + (1 - \lambda)x_j, \quad \tilde{y}_{ij} = \lambda \tilde{y}_i + (1 - \lambda)\tilde{y}_j. \quad (10)$$

The corresponding clean mixed label is  $\tilde{y}_{ij}^* = \lambda y_i^* + (1 - \lambda)y_j^*$ . The amount of corrupted-label mass in the mixed target is  $m_{ij}(\lambda) = \lambda r_i + (1 - \lambda)r_j$ . For MIXSV, since  $\mathbb{E}[\lambda] = p_{ij}$ , the expected corrupted-label mass is

$$M_{ij}^{\text{SV}} = p_{ij}r_i + (1 - p_{ij})r_j.$$

For standard Mixup,  $\lambda \sim \text{Beta}(\alpha, \alpha)$  and hence  $\mathbb{E}[\lambda] = 1/2$ . Its expected corrupted-label mass is

$$M_{ij}^{\text{Mix}} = \frac{1}{2}r_i + \frac{1}{2}r_j.$$

We use the following value-noise alignment condition.

**Assumption V.2.** For a pair  $(i, j)$ , the shifted Shapley values are aligned with label cleanliness:

$$(a_i^{(ij)} - a_j^{(ij)}) (r_i - r_j) \leq 0. \quad (11)$$

This condition means that a clean point receives a value no smaller than a corrupted point in the same pair. It captures the intended behavior of Shapley values computed from a clean validation set, since mislabeled points tend to hurt validation performance and therefore receive smaller values.

**Theorem V.2.** Suppose Assumption V.2 holds for a pair  $(i, j)$ . Then MIXSV assigns no larger expected corrupted-label mass than standard Mixup  $M_{ij}^{SV} \leq M_{ij}^{\text{Mix}}$ . More precisely,

$$M_{ij}^{\text{Mix}} - M_{ij}^{SV} = \frac{|a_i^{(ij)} - a_j^{(ij)}|}{2(a_i^{(ij)} + a_j^{(ij)})} |r_i - r_j|. \quad (12)$$

*Proof.* By the definition of  $p_{ij}$ ,

$$p_{ij} = \frac{a_i^{(ij)}}{a_i^{(ij)} + a_j^{(ij)}}.$$

Therefore,

$$p_{ij} - \frac{1}{2} = \frac{a_i^{(ij)} - a_j^{(ij)}}{2(a_i^{(ij)} + a_j^{(ij)})}.$$

The difference between the expected corrupted-label mass of MIXSV and that of standard Mixup is

$$\begin{aligned} M_{ij}^{SV} - M_{ij}^{\text{Mix}} &= (p_{ij}r_i + (1 - p_{ij})r_j) - \left(\frac{1}{2}r_i + \frac{1}{2}r_j\right) \\ &= \left(p_{ij} - \frac{1}{2}\right)(r_i - r_j) \\ &= \frac{(a_i^{(ij)} - a_j^{(ij)})(r_i - r_j)}{2(a_i^{(ij)} + a_j^{(ij)})}. \end{aligned}$$

By Assumption V.2, the numerator is non-positive. Thus,

$$M_{ij}^{SV} - M_{ij}^{\text{Mix}} \leq 0,$$

which proves

$$M_{ij}^{SV} \leq M_{ij}^{\text{Mix}}.$$

The exact improvement follows from taking the absolute value under the alignment condition:

$$\begin{aligned} M_{ij}^{\text{Mix}} - M_{ij}^{SV} &= \left|p_{ij} - \frac{1}{2}\right| |r_i - r_j| \\ &= \frac{|a_i^{(ij)} - a_j^{(ij)}|}{2(a_i^{(ij)} + a_j^{(ij)})} |r_i - r_j|. \end{aligned}$$

This completes the proof.  $\square$

Theorem V.2 shows that MIXSV reduces the expected contribution of corrupted labels whenever Shapley values rank clean points above corrupted points. The gain is zero when both endpoints have the same corruption status. The gain is largest when one endpoint is clean, the other endpoint is corrupted, and their Shapley values are well separated.

The following corollary gives a more explicit bound for clean-corrupted pairs.

**Corollary V.1.** Consider a pair that contains one clean point and one corrupted point. Let  $a_{\text{clean}}$  and  $a_{\text{noise}}$  be their shifted Shapley values. If  $\frac{a_{\text{clean}}}{a_{\text{noise}}} \geq \kappa > 1$ , then the expected corrupted-label mass under MIXSV satisfies  $M_{ij}^{SV} \leq \frac{1}{1+\kappa}$ , while

standard Mixup gives  $M_{ij}^{\text{Mix}} = \frac{1}{2}$ . Therefore, the reduction is at least

$$M_{ij}^{\text{Mix}} - M_{ij}^{SV} \geq \frac{\kappa - 1}{2(1 + \kappa)}. \quad (13)$$

*Proof.* For a clean-corrupted pair, the corrupted-label mass is exactly the interpolation weight assigned to the corrupted endpoint. Under MIXSV, this expected weight is

$$\frac{a_{\text{noise}}}{a_{\text{clean}} + a_{\text{noise}}}.$$

Using  $a_{\text{clean}}/a_{\text{noise}} \geq \kappa$ , we obtain

$$\frac{a_{\text{noise}}}{a_{\text{clean}} + a_{\text{noise}}} = \frac{1}{1 + a_{\text{clean}}/a_{\text{noise}}} \leq \frac{1}{1 + \kappa}.$$

Standard Mixup assigns expected weight  $1/2$  to either endpoint. Thus,

$$M_{ij}^{\text{Mix}} - M_{ij}^{SV} \geq \frac{1}{2} - \frac{1}{1 + \kappa} = \frac{\kappa - 1}{2(1 + \kappa)}.$$

This completes the proof.  $\square$

We can further translate the corrupted-label mass into a bound on the perturbation of the training loss. Assume that the loss  $\ell(f(x), y)$  is  $L$ -Lipschitz in the label argument under the  $\ell_1$  norm

$$|\ell(f(x), y) - \ell(f(x), y')| \leq L\|y - y'\|_1.$$

Then

$$\begin{aligned} \|\tilde{y}_{ij} - \tilde{y}_{ij}^*\|_1 &= \|\lambda(\tilde{y}_i - y_i^*) + (1 - \lambda)(\tilde{y}_j - y_j^*)\|_1 \\ &\leq \lambda\|\tilde{y}_i - y_i^*\|_1 + (1 - \lambda)\|\tilde{y}_j - y_j^*\|_1 \\ &\leq 2(\lambda r_i + (1 - \lambda)r_j). \end{aligned}$$

Taking expectation over  $\lambda$  gives

$$\mathbb{E}_\lambda [\|\tilde{y}_{ij} - \tilde{y}_{ij}^*\|_1] \leq 2M_{ij}^{SV}.$$

Therefore,

$$\mathbb{E}_\lambda [\ell(f(\tilde{x}_{ij}), \tilde{y}_{ij}) - \ell(f(\tilde{x}_{ij}), \tilde{y}_{ij}^*)] \leq 2LM_{ij}^{SV} \leq 2LM_{ij}^{\text{Mix}}.$$

This result links the pairwise weighting rule of MIXSV to robustness against label noise. When a clean validation set assigns smaller Shapley values to mislabeled points, MIXSV reduces their expected weight in both the mixed input and the mixed target. Thus, noisy labels have a smaller effect on the augmented training objective than they do under standard Mixup.

### C. Local Ordering of kNN-Shapley Values

The previous two subsections rely on the ordering induced by Shapley values. We now show that the kNN-Shapley estimator used in MIXSV naturally produces the desired local ordering around clean validation samples.

Consider a clean validation sample  $v = (x_v, y_v)$ . Let  $\alpha_1(v), \alpha_2(v), \dots, \alpha_n(v)$  be the indices of training samples sorted by increasing distance to  $x_v$ . For simplicity, we assume there are no distance ties. Ties can be handled by any fixed

tie-breaking rule. Define the label-match indicator  $g_\ell(v) = \mathbb{1}[\bar{y}_{\alpha_\ell}(v) = y_v]$ , where  $\bar{y}_i$  is the observed label of training point  $x_i$ . For notational simplicity, we omit the dependence on  $v$  when it is clear from context and write  $\alpha_\ell$  and  $g_\ell$ .

For a fixed validation point  $v$ , the kNN-Shapley score satisfies the recursion  $s_{\alpha_n}(v) = \frac{g_n(v)}{n}$ , and for  $\ell = n-1, \dots, 1$ ,

$$s_{\alpha_\ell}(v) = s_{\alpha_{\ell+1}}(v) + (g_\ell(v) - g_{\ell+1}(v)) w_\ell,$$

where  $w_\ell = \frac{1}{K} \frac{\min\{K, \ell\}}{\ell}$ . The final kNN-Shapley value is the average over the validation set

$$\mathcal{SV}_i = \frac{1}{|D_v|} \sum_{v \in D_v} s_i(v).$$

The sequence  $\{w_\ell\}_{\ell=1}^{n-1}$  is non-increasing, since  $w_\ell = 1/K$  for  $\ell \leq K$  and  $w_\ell = 1/\ell$  for  $\ell > K$ . This monotonicity is the key reason why closer label-matching points receive no smaller kNN-Shapley scores.

**Lemma V.1.** *Fix a validation point  $v = (x_v, y_v)$ . Let  $r < s$ . If the two training points at ranks  $r$  and  $s$  both match the validation label, namely  $g_r(v) = g_s(v) = 1$ , then*

$$s_{\alpha_r}(v) \geq s_{\alpha_s}(v). \quad (14)$$

*Proof.* From the kNN-Shapley recursion,

$$s_{\alpha_r}(v) - s_{\alpha_s}(v) = \sum_{\ell=r}^{s-1} (g_\ell(v) - g_{\ell+1}(v)) w_\ell.$$

Using summation by parts, we obtain

$$\begin{aligned} s_{\alpha_r}(v) - s_{\alpha_s}(v) &= g_r(v)w_r + \sum_{\ell=r+1}^{s-1} g_\ell(v)(w_\ell - w_{\ell-1}) - g_s(v)w_{s-1}. \end{aligned}$$

Since  $w_\ell$  is non-increasing, we have  $w_\ell - w_{\ell-1} \leq 0$ . Also,  $0 \leq g_\ell(v) \leq 1$ . Therefore,

$$g_\ell(v)(w_\ell - w_{\ell-1}) \geq w_\ell - w_{\ell-1}.$$

Using  $g_r(v) = g_s(v) = 1$ , we get

$$\begin{aligned} s_{\alpha_r}(v) - s_{\alpha_s}(v) &\geq w_r + \sum_{\ell=r+1}^{s-1} (w_\ell - w_{\ell-1}) - w_{s-1} \\ &= 0. \end{aligned}$$

Thus,  $s_{\alpha_r}(v) \geq s_{\alpha_s}(v)$ . This completes the proof.  $\square$

Lemma V.1 states that, among training points that match the clean validation label, a closer point receives no smaller local kNN-Shapley contribution. This directly supports the value-discrepancy alignment condition in Assumption V.1.

We next show a stronger result for the clean-versus-corrupted case. A corrupted point often has an observed label that does not match nearby clean validation samples. In this case, kNN-Shapley assigns a larger local contribution to the clean label-matching point.

**Lemma V.2.** *Fix a validation point  $v = (x_v, y_v)$ . Let  $r < s$ . If the closer point matches the validation label and the farther point does not, namely  $g_r(v) = 1$ ,  $g_s(v) = 0$ , then*

$$s_{\alpha_r}(v) - s_{\alpha_s}(v) \geq w_{s-1} > 0. \quad (15)$$

*Proof.* Using the same expansion as in Lemma V.1,

$$s_{\alpha_r}(v) - s_{\alpha_s}(v) = g_r(v)w_r + \sum_{\ell=r+1}^{s-1} g_\ell(v)(w_\ell - w_{\ell-1}) - g_s(v)w_{s-1}.$$

Since  $g_r(v) = 1$  and  $g_s(v) = 0$ , this becomes

$$s_{\alpha_r}(v) - s_{\alpha_s}(v) = w_r + \sum_{\ell=r+1}^{s-1} g_\ell(v)(w_\ell - w_{\ell-1}).$$

Again, because  $w_\ell - w_{\ell-1} \leq 0$  and  $0 \leq g_\ell(v) \leq 1$ ,

$$\begin{aligned} s_{\alpha_r}(v) - s_{\alpha_s}(v) &\geq w_r + \sum_{\ell=r+1}^{s-1} (w_\ell - w_{\ell-1}) \\ &= w_{s-1}. \end{aligned}$$

Since  $w_{s-1} > 0$ , the result follows.  $\square$

Lemma V.2 shows that a label-matching point that is closer to a clean validation sample receives a strictly larger local contribution than a farther mismatched point. This explains why label-flipped samples tend to receive smaller kNN-Shapley values when the validation set is clean.

Finally, we connect the kNN-Shapley ordering to the interpolation weights used by MIXSV. Recall that, for a MIXSV pair  $(i, j)$ , the shifted Shapley values are denoted by  $a_i^{(ij)}$  and  $a_j^{(ij)}$ , where the same pairwise shift is added to both original Shapley values. Therefore, the shift preserves their order:

$$a_i^{(ij)} - a_j^{(ij)} = \mathcal{SV}_i - \mathcal{SV}_j.$$

**Proposition V.1.** *For any MIXSV pair  $(i, j)$ , the expected interpolation weight satisfies  $p_{ij} - \frac{1}{2} = \frac{\mathcal{SV}_i - \mathcal{SV}_j}{2(a_i^{(ij)} + a_j^{(ij)})}$ . Consequently,  $\mathcal{SV}_i \geq \mathcal{SV}_j \iff p_{ij} \geq \frac{1}{2}$ .*

*Proof.* By the definition of MIXSV,  $p_{ij} = \frac{a_i^{(ij)}}{a_i^{(ij)} + a_j^{(ij)}}$ . Thus,

$$p_{ij} - \frac{1}{2} = \frac{a_i^{(ij)}}{a_i^{(ij)} + a_j^{(ij)}} - \frac{1}{2} = \frac{a_i^{(ij)} - a_j^{(ij)}}{2(a_i^{(ij)} + a_j^{(ij)})}.$$

Since the pairwise shift preserves value differences,

$$a_i^{(ij)} - a_j^{(ij)} = \mathcal{SV}_i - \mathcal{SV}_j.$$

Substituting this equality gives the result.  $\square$

Proposition V.1 shows that the ordering produced by kNN-Shapley is directly transferred to the expected interpolation weights in MIXSV. Combining this proposition with Lemma V.1, same-label points that are closer to clean validation samples receive larger expected interpolation weights. This gives the mechanism behind the validation-discrepancy reduction in Theorem V.1.

Combining Proposition V.1 with Lemma V.2, clean label-matching points receive larger expected interpolation weights than nearby mislabeled points. This gives the mechanism behind the label-noise attenuation result in Theorem V.2. Therefore, kNN-Shapley provides a local ranking principle that connects clean validation performance, sample quality, and the adaptive interpolation rule of MIXSV.

## VI. EMPIRICAL VERIFICATION

In this section, we evaluate MIXSV under several experimental settings, including standard classification, adversarial robustness, data poisoning, hyper-parameter sensitivity, and experiments with Monte Carlo-based Shapley values. Section VI-A describes the datasets, model architectures, baselines, and experimental protocol. The following subsections present the main empirical results and additional analyses, including the comparison between Monte Carlo and kNN Shapley estimators and the study of surrogate models.

### A. Experimental Setup

To evaluate MIXSV, we conducted experiments across various datasets and scenarios, comparing it with standard training (no Mixup), traditional Mixup, and Manifold Mixup. The evaluation spans classification tasks, adversarial robustness, and long-tailed distributions. Below, we detail the datasets, model architectures, baseline methods, and experimental protocol.

**Datasets** We evaluate MIXSV on five datasets, focusing primarily on tabular data due to the current limitations of kNN Shapley, which can only compute Shapley values for tabular datasets. The datasets include a synthetic dataset with 500 data points and 50 numerical features, and the **Adult** dataset [24], which contains 32,561 samples with 14 features (continuous attributes like age and hours worked per week, as well as categorical ones like education and occupation) for binary classification tasks predicting income levels. We also use the **Arrhythmia** dataset [25], consisting of 452 medical records with 279 features for heart arrhythmia classification. For image data, we extract deep features using ResNet-50 [26] on **CIFAR-100**, which contains 60,000 images across 100 classes, and its long-tail version, where class distributions follow a power-law distribution to simulate real-world imbalances.

**Model** The neural network for tabular data architecture used in this study is fully-connected, consisting of two hidden layers with 64 ReLU units each. The input layer maps the input dimension to 64 hidden units, followed by a ReLU activation function to introduce non-linearity. This process is repeated in the second hidden layer. The final output layer maps the 64 hidden units to the output dimension. The model parameters are optimized using the Adam optimizer [27] with default hyperparameters, trained over 100 epochs using mini-batches of size 64.

**Baselines** We compare MIXSV against the following baselines:

- No Mixup: Standard training without any data augmentation.
- Mixup [2]: Vanilla Mixup with  $\lambda \sim \text{Beta}(\alpha, \alpha)$ .

- Manifold Mixup [11]: Performs Mixup at intermediate feature layers.
- AdaMixup [12]: Adaptive version of Mixup where the mixing policies are learned dynamically from the data using an additional network and objective function

**Methods** Following the practice of Ren et al. [28], we assume access to a clean validation set to compute Shapley values. For datasets with an official test set, we randomly sample 30% of the test data as the validation set and retain the remaining 70% as the final test set. For datasets without an official test set, we split the data into training, validation, and test sets in a 7:1:2 ratio. This use of a validation set to guide training is widely accepted in the machine learning community, similar to Ren et al., who reweight training samples using gradients from a validation set, we leverage the validation set to compute Shapley values that reflect data importance. Since the validation set is not directly used for optimization but only serves to evaluate marginal contributions, it does not constitute data leakage.

TABLE III: Comparison of computation time and accuracy for Shapley Values. We do not compute the accuracy when the estimated runtime is excessively long.

| Dataset   | Task | Method                                   | Time / Accuracy(%)           |
|-----------|------|------------------------------------------|------------------------------|
| synthetic | LR   | Monte Carlo (10k samples)<br>kNN Shapley | 87h / –<br>< 1s / 63.80      |
|           | SVM  | Monte Carlo (10k samples)<br>kNN Shapley | 469s / 74.70<br>< 1s / 71.50 |
| Adult     | LR   | Monte Carlo (10k samples)<br>kNN Shapley | > 600h / –<br>104s / 84.11   |
|           | SVM  | Monte Carlo<br>kNN Shapley               | > 1000h / –<br>104s / 85.19  |

### B. Efficiency

Before evaluating MIXSV, we first compare the time consumption and accuracy of using sampling estimation and kNN Shapley to calculate Shapley values. We use LR and SVM to perform classification tasks on a synthetic and the Adult datasets. As shown in Table III. Using kNN Shapley achieves similar task accuracy as Monte Carlo sampling but requires significantly less time. Therefore, it can be concluded that when it comes to large datasets or neural networks, only kNN Shapley is applicable and can achieve effective results.

### C. Standard Classification Performance

To verify the effectiveness of the proposed method, we conducted comparative experiments on four representative datasets (synthetic, Adult, CIFAR-100, and Arrhythmia) to compare the performance of Mixup, MIXSV, no Mixup, and Manifold Mixup on classification tasks. Each dataset was trained with the same network structure and training hyperparameters, and the experiment was repeated multiple times to reduce the influence of random factors. Figure 4 shows the change of test accuracy (upper row) and test loss (lower row) with the training round (epoch), and the shaded part represents

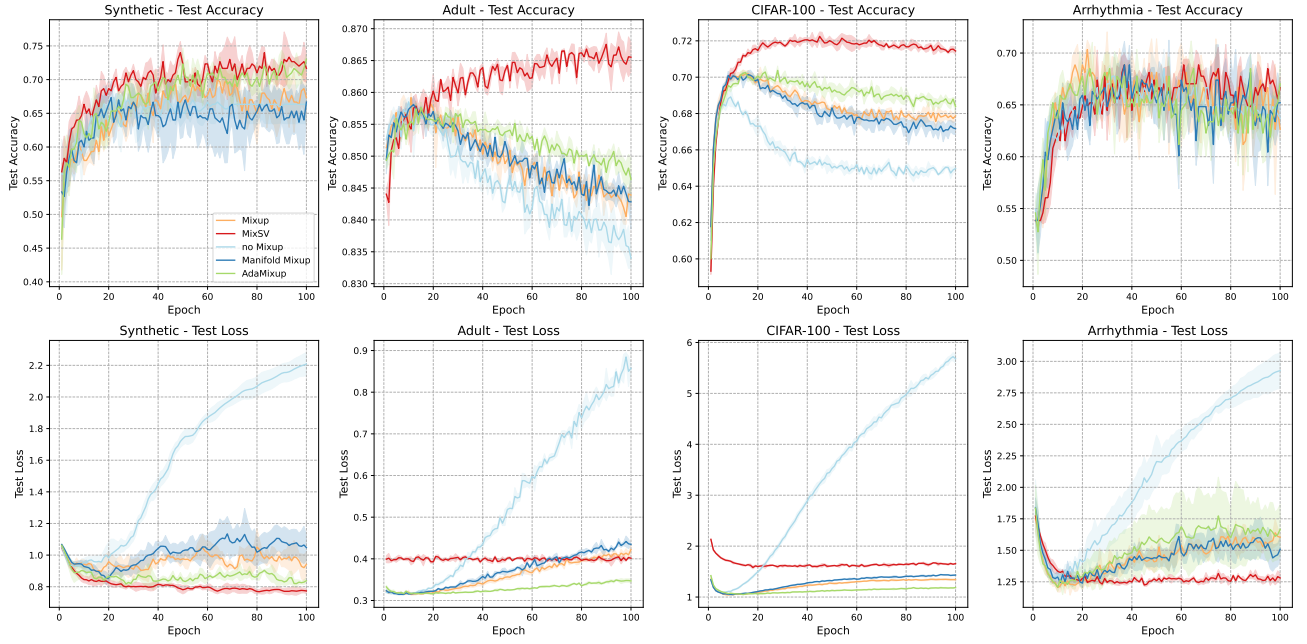


Fig. 4: Standard Classification Performance

the standard deviation range of the experimental results with different random seeds.

Across all datasets, MIXSV consistently achieves the highest accuracy and the most stable loss during training. For example, on the Synthetic and CIFAR-100 datasets, MIXSV rapidly surpasses baseline methods and maintains its advantage throughout training. On the Synthetic and Arrhythmia datasets, MIXSV not only achieves the highest final accuracy, but also obtains the lowest loss during training. These results demonstrate that MIXSV provides more robust generalization and better resistance to overfitting compared to standard Mixup and its variants. Overall, the empirical evidence highlights MIXSV as a reliable data augmentation strategy for diverse classification tasks.

#### D. Empirical Distribution Alignment

Motivated by the validation-discrepancy analysis in Section V-A, we further examine whether MixSV improves the alignment between the augmented training distribution and the held-out test distribution. Following prior work that relates distribution discrepancy to model performance [29], [30], we measure the conditional Wasserstein distance, which accounts for label-dependent distribution differences. Since Mixup produces soft labels, we conduct this analysis using label-consistent pairs, where Mixup is applied only to training samples from the same class.

As shown in Figure 5, MixSV achieves smaller conditional Wasserstein distances than both the original training set and standard Mixup. The MixSV points also correspond to higher test accuracy, which is consistent with our theoretical analysis.

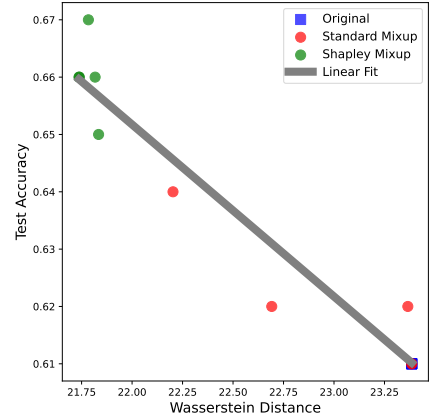


Fig. 5: Relationship between conditional Wasserstein distance and test accuracy. MixSV achieves lower distribution discrepancy and higher accuracy than standard Mixup and the original training set.

#### E. Robustness Against Adversarial Attacks

To investigate the robustness of our models against adversarial perturbations, we evaluated four mixup strategies—No Mixup, Standard Mixup, Adamixup, Manifold Mixup, and MIXSV—under two common  $\ell_\infty$ -based attack scenarios: Fast Gradient Sign Method (FGSM) [31], Projected Gradient Descent (PGD) [32] and Auto Attack [33]. These attacks introduce controlled perturbations to input data points, targeting the model’s decision boundaries. Specifically, we conducted experiments on four diverse datasets and measured each model’s accuracy at three levels of adversarial strength, controlled by the perturbation parameter  $\epsilon \in \{0.001, 0.01, 0.1\}$ . These val-

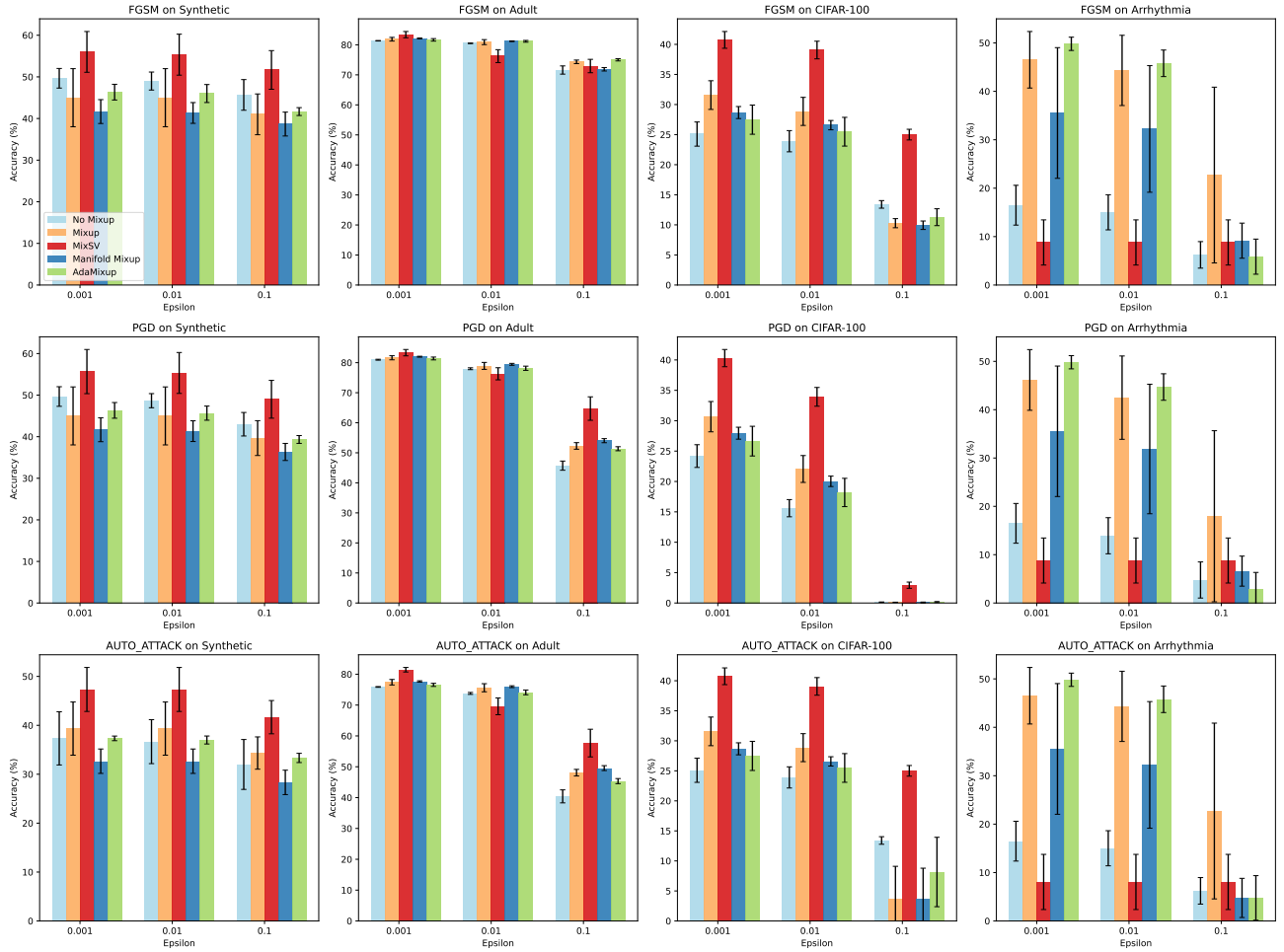


Fig. 6: Robustness Against Adversarial Attacks

ues represent progressively higher magnitudes of adversarial noise added to the input, thereby enabling a comprehensive comparison of each mixup method’s resilience.

The results, depicted in Figure 6, reveal distinct trends in model performance across different mixup methods and adversarial perturbation intensities. Under FGSM attacks, MIXSV demonstrated consistent superiority over other techniques at higher epsilon values across most datasets, achieving particularly significant performance margins in CIFAR-100. However, its effectiveness diminished notably on the Arrhythmia dataset. Similar patterns emerged with PGD attacks, where MIXSV maintained enhanced resilience against adversarial noise, especially at epsilon = 0.1. The margin of improvement varied across datasets, suggesting that MIXSV’s effectiveness might be particularly tuned to the underlying data distribution and feature space.

#### F. Robustness Against Data Poisoning

Table IV reports the top-1 accuracy (%) when a fraction of the training labels is flipped at random. **MIXSV is consistently the most resilient method.** At 20% noise it still retains 90–96% of its clean-set accuracy on every benchmark,

whereas the strongest alternative (AdaMixup) loses between 7 and 26 percentage points (e.g., Adult:  $-3.35$  pp; CIFAR-100:  $-6.16$  pp). On SYNTHETIC, MIXSV’s drop is only 6.67 pp, versus 16 pp for standard Mixup; on CIFAR-100 it keeps **68.79%** accuracy—an **11.1 pp** absolute gain over standard Mixup. The sole outlier is ARRHYTHMIA at 10% noise, where AdaMixup edges ahead by 1.1 pp, but MIXSV regains a clear lead once noise exceeds this level. These results show that weighting pairs by Shapley value largely immunises Mixup against label noise, yielding marked improvements in robustness across tabular and image domains.

#### G. Hyper-parameter Sensitivity

We examine MIXSV’s robustness with respect to its two key hyper-parameters: the number of neighbours  $K$  used by kNN-Shapley and the Mixup shape parameter  $\alpha$  that controls the overall strength of interpolation.

a) *Effect of  $K$ .* We ran MIXSV with  $K \in \{1, 3, 5, 10\}$  on four datasets; the results in Table VII show only minor accuracy fluctuations. While certain datasets like ‘synthetic’ and ‘arrhythmia’ show peak performance at  $K = 3$  or  $K = 10$ ,

TABLE IV: Comparison of Mixup methods under different label flipping ratios.

| Dataset    | Method         | Flip ratio   |                                             |                                             |
|------------|----------------|--------------|---------------------------------------------|---------------------------------------------|
|            |                | 0%           | 10%                                         | 20%                                         |
| Synthetic  | No Mixup       | 66.33        | 57.67 ( $\downarrow 8.66$ )                 | 51.00 ( $\downarrow 15.33$ )                |
|            | Standard Mixup | 66.33        | 61.00 ( $\downarrow 5.33$ )                 | 50.33 ( $\downarrow 16$ )                   |
|            | Manifold Mixup | 63.00        | 57.00 ( $\downarrow 6$ )                    | 53.00 ( $\downarrow 10$ )                   |
|            | Adamixup       | 69.67        | 60.67 ( $\downarrow 9$ )                    | 51.33 ( $\downarrow 18.34$ )                |
|            | MIXSV          | <b>71.00</b> | <b>67.00 (<math>\downarrow 4</math>)</b>    | <b>64.33 (<math>\downarrow 6.67</math>)</b> |
| Adult      | No Mixup       | 83.71        | 81.36 ( $\downarrow 2.35$ )                 | 74.64 ( $\downarrow 9.07$ )                 |
|            | Standard Mixup | 84.22        | 82.27 ( $\downarrow 1.95$ )                 | 78.40 ( $\downarrow 5.82$ )                 |
|            | Manifold Mixup | 84.44        | 81.96 ( $\downarrow 2.48$ )                 | 78.53 ( $\downarrow 5.91$ )                 |
|            | Adamixup       | 84.76        | 83.51 ( $\downarrow 1.25$ )                 | 81.41 ( $\downarrow 3.35$ )                 |
|            | MIXSV          | <b>86.61</b> | <b>86.31 (<math>\downarrow 0.3</math>)</b>  | <b>84.79 (<math>\downarrow 1.82</math>)</b> |
| CIFAR-100  | No Mixup       | 64.32        | 54.87 ( $\downarrow 9.45$ )                 | 46.48 ( $\downarrow 17.84$ )                |
|            | Standard Mixup | 68.10        | 62.42 ( $\downarrow 5.68$ )                 | 57.69 ( $\downarrow 10.41$ )                |
|            | Manifold Mixup | 67.34        | 60.81 ( $\downarrow 6.53$ )                 | 56.03 ( $\downarrow 11.31$ )                |
|            | Adamixup       | 68.68        | 65.12 ( $\downarrow 3.56$ )                 | 62.52 ( $\downarrow 6.16$ )                 |
|            | MIXSV          | <b>71.30</b> | <b>70.36 (<math>\downarrow 0.94</math>)</b> | <b>68.79 (<math>\downarrow 2.51</math>)</b> |
| Arrhythmia | No Mixup       | 62.27        | 61.53 ( $\downarrow 0.74$ )                 | 52.01 ( $\downarrow 10.26$ )                |
|            | Standard Mixup | 63.37        | 59.71 ( $\downarrow 3.66$ )                 | 47.25 ( $\downarrow 16.12$ )                |
|            | Manifold Mixup | 64.83        | 60.07 ( $\downarrow 4.76$ )                 | 53.84 ( $\downarrow 10.99$ )                |
|            | Adamixup       | 61.53        | 62.63 ( $\uparrow 1.1$ )                    | 53.47 ( $\downarrow 8.06$ )                 |
|            | MIXSV          | <b>67.76</b> | <b>65.20 (<math>\downarrow 2.56</math>)</b> | <b>64.46 (<math>\downarrow 3.3</math>)</b>  |

the differences are generally small, indicating stable behavior of the method under different neighborhood sizes.

#### H. Using Monte Carlo-based Shapley Value

We also report results on Fashion-MNIST (fMNIST) [34] using Data Shapley computed with the CC method [35], summarized in Table V. This experiment differs from those in the main text in two key aspects: (1) both the Shapley value computation and the subsequent training are performed via an SVM model (rather than the kNN or neural network approaches adopted previously for quicker computation), and (2) to mitigate the high computational cost of CC-based Data Shapley, we restrict the dataset to the first 2,000 data points of fMNIST. Despite the reduced dataset size, these results illustrate the effectiveness of MIXSV under a different model and Shapley value estimation strategy.

TABLE V: Experimental results for FMNIST using Data Shapley calculated by CC [35].

| Method   | Top-1 Accuracy |
|----------|----------------|
| No Mixup | 0.795          |
| Mixup    | 0.800          |
| MIXSV    | <b>0.812</b>   |

For the CIFAR-10 dataset, we applied a series of preprocessing steps to prepare the data for training with the SVM model. We still used 2000 data points for the experiment and calculated the Shapley value by CC. First, each image was flattened into a one-dimensional vector and normalized to the [0,1] range. Specifically, the pixel values were divided by 255. Next, the labels were reshaped into a one-dimensional array to match the expected input format for the model.

To further reduce computational complexity and extract meaningful features, we applied Principal Component Analysis (PCA) to the flattened image vectors. The dimensionality of the data was reduced to 100 principal components, making the data easier for SVM training and evaluation.

TABLE VI: Experimental results for CIFAR10 using Data Shapley calculated by CC [35].

| Method   | Top-1 Accuracy | Top-3 Accuracy |
|----------|----------------|----------------|
| No Mixup | 0.294          | 0.628          |
| Mixup    | 0.353          | 0.695          |
| MIXSV    | <b>0.418</b>   | <b>0.722</b>   |

TABLE VII: Effect of  $K$  on classification accuracy (%).

| Dataset    | $K = 1$ | $K = 3$      | $K = 5$      | $K = 10$     |
|------------|---------|--------------|--------------|--------------|
| cifar100   | 70.44   | 71.20        | <b>71.30</b> | 71.11        |
| adult      | 86.09   | 86.60        | <b>86.61</b> | 86.42        |
| arrhythmia | 63.74   | 67.03        | 67.76        | <b>68.13</b> |
| synthetic  | 72.00   | <b>74.00</b> | 71.00        | 71.00        |

a) *Effect of  $\alpha$ .* Sweeping  $\alpha \in \{0.2, 0.4, 0.8, 1.0\}$  for MIXSV yields the accuracies in Table VIII. Again, differences are modest, we therefore select the best-performing value  $\alpha = 1$  for all subsequent experiments. Overall, both tables confirm that MIXSV maintains strong performance across a wide range of hyperparameter choices, simplifying practical deployment.

TABLE VIII: Effect of  $\alpha$  on classification accuracy (%).

| Dataset    | $\alpha = 0.2$ | $\alpha = 0.4$ | $\alpha = 0.8$ | $\alpha = 1.0$ |
|------------|----------------|----------------|----------------|----------------|
| cifar100   | 68.54          | 69.04          | 70.63          | <b>71.30</b>   |
| adult      | 86.59          | 86.51          | 86.47          | <b>86.61</b>   |
| arrhythmia | <b>68.13</b>   | <b>68.13</b>   | 67.03          | 67.76          |
| synthetic  | 65.00          | 70.00          | <b>72.00</b>   | 71.00          |

## VII. CONCLUSION AND LIMITATIONS

In this paper, we propose MIXSV, a novel data augmentation framework that integrates Shapley values into the Mixup procedure to address uniform data point weighting limitations. By adaptively assigning higher interpolation weights to data points with greater Shapley values, MIXSV improves classification accuracy, robustness, and distribution alignment, as demonstrated across various datasets, including long-tailed and adversarial scenarios.

While the exact computation of Shapley values can be expensive in large-scale settings, our use of kNN Shapley highlights a promising route for balancing precision and efficiency. Future work will explore alternative approximations of Shapley values (e.g., sampling-based or model-specific methods), and the extension of MIXSV to additional modalities such as text, image, and multimodal tasks. Overall, MIXSV demonstrates a principled and effective approach to Mixup that respects the inherent value of individual training data points, offering a new perspective for building more robust and generalizable machine learning models.

## REFERENCES

- [1] T. Devries and G. W. Taylor, "Improved regularization of convolutional neural networks with cutout," *CoRR*, vol. abs/1708.04552, 2017. [Online]. Available: <http://arxiv.org/abs/1708.04552>
- [2] H. Zhang, M. Cissé, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," in *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018. [Online]. Available: <https://openreview.net/forum?id=r1Ddp1-Rb>
- [3] S. Yun, D. Han, S. Chun, S. J. Oh, Y. Yoo, and J. Choe, "Cutmix: Regularization strategy to train strong classifiers with localizable features," in *2019 IEEE/CVF International Conference on Computer Vision, ICCV 2019, Seoul, Korea (South), October 27 - November 2, 2019*. IEEE, 2019, pp. 6022–6031. [Online]. Available: <https://doi.org/10.1109/ICCV.2019.00612>
- [4] L. Zhang, Z. Deng, K. Kawaguchi, A. Ghorbani, and J. Zou, "How does mixup help with robustness and generalization?" in *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. [Online]. Available: <https://openreview.net/forum?id=8yKEo06dKN0>
- [5] L. S. Shapley, "A value for n-person games," *Contributions to the Theory of Games*, vol. 2, no. 28, pp. 307–317, 1953.
- [6] A. Ghorbani and J. Y. Zou, "Data shapley: Equitable valuation of data for machine learning," in *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 2019, pp. 2242–2251. [Online]. Available: <http://proceedings.mlr.press/v97/ghorbani19c.html>
- [7] R. Jia, D. Dao, B. Wang, F. A. Hubis, N. Hynes, N. M. Gürel, B. Li, C. Zhang, D. Song, and C. J. Spanos, "Towards efficient data valuation based on the shapley value," in *The 22nd International Conference on Artificial Intelligence and Statistics, AISTATS 2019, 16-18 April 2019, Naha, Okinawa, Japan*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and M. Sugiyama, Eds., vol. 89. PMLR, 2019, pp. 1167–1176. [Online]. Available: <http://proceedings.mlr.press/v89/jia19a.html>
- [8] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *J. Big Data*, vol. 6, p. 60, 2019. [Online]. Available: <https://doi.org/10.1186/s40537-019-0197-0>
- [9] S. Yun, D. Han, S. Chun, S. J. Oh, Y. Yoo, and J. Choe, "Cutmix: Regularization strategy to train strong classifiers with localizable features," in *2019 IEEE/CVF International Conference on Computer Vision, ICCV 2019, Seoul, Korea (South), October 27 - November 2, 2019*. IEEE, 2019, pp. 6022–6031. [Online]. Available: <https://doi.org/10.1109/ICCV.2019.00612>
- [10] D. Hendrycks, N. Mu, E. D. Cubuk, B. Zoph, J. Gilmer, and B. Lakshminarayanan, "Augmix: A simple data processing method to improve robustness and uncertainty," in *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. [Online]. Available: <https://openreview.net/forum?id=SlgmrxHFvB>
- [11] V. Verma, A. Lamb, C. Beckham, A. Najafi, I. Mitliagkas, D. Lopez-Paz, and Y. Bengio, "Manifold mixup: Better representations by interpolating hidden states," in *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 2019, pp. 6438–6447. [Online]. Available: <http://proceedings.mlr.press/v97/verma19a.html>
- [12] H. Guo, Y. Mao, and R. Zhang, "Mixup as locally linear out-of-manifold regularization," in *The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, The Thirty-First Innovative Applications of Artificial Intelligence Conference, IAAI 2019, The Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2019, Honolulu, Hawaii, USA, January 27 - February 1, 2019*. AAAI Press, 2019, pp. 3714–3722. [Online]. Available: <https://doi.org/10.1609/aaai.v33i01.33013714>
- [13] M. Kang and S. Kim, "Guidedmixup: An efficient mixup strategy guided by saliency maps," in *Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023, Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI 2023, Thirteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2023, Washington, DC, USA, February 7-14, 2023*, B. Williams, Y. Chen, and J. Neville, Eds. AAAI Press, 2023, pp. 1096–1104. [Online]. Available: <https://doi.org/10.1609/aaai.v37i1.25191>
- [14] J. Kim, W. Choo, and H. O. Song, "Puzzle mix: Exploiting saliency and local statistics for optimal mixup," in *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 5275–5285. [Online]. Available: <http://proceedings.mlr.press/v119/kim20b.html>
- [15] A. F. M. S. Uddin, M. S. Monira, W. Shin, T. Chung, and S. Bae, "Saliencymix: A saliency guided data augmentation strategy for better regularization," in *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. [Online]. Available: <https://openreview.net/forum?id=M0QkvBGTtQ>
- [16] S. M. Lundberg and S. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, I. Guyon, U. von Luxburg, S. Bengio, H. M. Wallach, R. Fergus, S. V. N. Vishwanathan, and R. Garnett, Eds., 2017, pp. 4765–4774.
- [17] Y. Tian, Y. Ding, S. Fu, and D. Liu, "Data boundary and data pricing based on the shapley value," *IEEE Access*, vol. 10, pp. 14 288–14 300, 2022. [Online]. Available: <https://doi.org/10.1109/ACCESS.2022.3147799>
- [18] D. V. Fryer, I. Strümke, and H. D. Nguyen, "Shapley values for feature selection: The good, the bad, and the axioms," *IEEE Access*, vol. 9, pp. 144 352–144 360, 2021. [Online]. Available: <https://doi.org/10.1109/ACCESS.2021.3119110>
- [19] L. Antwarg, C. Galed, N. Shimoni, L. Rokach, and B. Shapira, "Shapley-based feature augmentation," *Inf. Fusion*, vol. 96, pp. 92–102, 2023. [Online]. Available: <https://doi.org/10.1016/j.inffus.2023.03.010>
- [20] S. Kwon and Y. Lee, "Explainability-based mix-up approach for text data augmentation," *ACM Trans. Knowl. Discov. Data*, vol. 17, no. 1, pp. 13:1–13:14, 2023. [Online]. Available: <https://doi.org/10.1145/3533048>
- [21] J. Zhang, L. Lin, and J. Liu, "Shap-mix: Shapley value guided mixing for long-tailed skeleton based action recognition," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI 2024, Jeju, South Korea, August 3-9, 2024*. ijcai.org, 2024, pp. 1688–1696. [Online]. Available: <https://www.ijcai.org/proceedings/2024/187>
- [22] X. Deng and C. H. Papadimitriou, "On the complexity of cooperative solution concepts," *Math. Oper. Res.*, vol. 19, no. 2, pp. 257–266, 1994. [Online]. Available: <https://doi.org/10.1287/moor.19.2.257>
- [23] R. Jia, D. Dao, B. Wang, F. A. Hubis, N. M. Gürel, B. Li, C. Zhang, C. J. Spanos, and D. Song, "Efficient task-specific data valuation for nearest neighbor algorithms," *Proc. VLDB Endow.*, vol. 12, no. 11, pp. 1610–1623, 2019. [Online]. Available: <http://www.vldb.org/pvldb/vol12/p1610-jia.pdf>
- [24] B. Becker and R. Kohavi, "Adult," UCI Machine Learning Repository, 1996, DOI: <https://doi.org/10.24432/C5XW20>.
- [25] H. A. Guvenir, B. Acar, G. Demiroz, and A. Cekin, "A supervised machine learning algorithm for arrhythmia analysis," in *Computers in Cardiology 1997*. IEEE, 1997, pp. 433–436.
- [26] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 2016, pp. 770–778. [Online]. Available: <https://doi.org/10.1109/CVPR.2016.90>
- [27] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2015. [Online]. Available: <http://arxiv.org/abs/1412.6980>
- [28] M. Ren, W. Zeng, B. Yang, and R. Urtasun, "Learning to reweight examples for robust deep learning," in *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholm, Sweden, July 10-15, 2018*, ser. Proceedings of Machine Learning Research, J. G. Dy and A. Krause, Eds., vol. 80. PMLR, 2018, pp. 4331–4340. [Online]. Available: <http://proceedings.mlr.press/v80/ren18a.html>
- [29] H. A. Just, F. Kang, T. Wang, Y. Zeng, M. Ko, M. Jin, and R. Jia, "LAVA: data valuation without pre-specified learning algorithms," in *The Eleventh International Conference on Learning Representations*,

- ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. [Online]. Available: <https://openreview.net/forum?id=JJuP86nBl4q>
- [30] S. Kessler, T. Le, and V. Nguyen, “SAVA: scalable learning-agnostic data valuation,” *CoRR*, vol. abs/2406.01130, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2406.01130>
  - [31] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” in *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2015. [Online]. Available: <http://arxiv.org/abs/1412.6572>
  - [32] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards deep learning models resistant to adversarial attacks,” in *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018. [Online]. Available: <https://openreview.net/forum?id=rJzIBfZAb>
  - [33] F. Croce and M. Hein, “Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks,” in *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 2206–2216. [Online]. Available: <http://proceedings.mlr.press/v119/croce20b.html>
  - [34] H. Xiao, K. Rasul, and R. Vollgraf, “Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms,” *CoRR*, vol. abs/1708.07747, 2017. [Online]. Available: <http://arxiv.org/abs/1708.07747>
  - [35] J. Zhang, Q. Sun, J. Liu, L. Xiong, J. Pei, and K. Ren, “Efficient sampling approaches to shapley value approximation,” *Proc. ACM Manag. Data*, vol. 1, no. 1, pp. 48:1–48:24, 2023. [Online]. Available: <https://doi.org/10.1145/3588728>